



早期Java组件化技术的缺陷

北京理工大学计算机学院
金旭亮

这事情，说来话长.....



任何一项技术，都不是凭空产生的，往往有它的历史渊源。

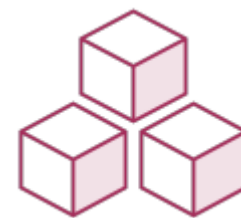
我们这门介绍Java模块化技术的课程，也需要从Java早期的组件化开发技术说起，才能把事情给讲清楚.....

JAR包是早期Java组件化开发的核心构件

JAR包实际上是一个采用zip标准构建出来的“压缩包”，里面包容了程序运行所需的.class文件和相关资源，通常，还附加有相应的元数据，放到META-INF文件夹中。



JAR包与Java应用程序



JAR包有名称，彼此之间存在着依赖关系，如果需要的话，每个JAR包中的公有类型，都能够被其他JAR包中的代码所访问。



启动应用程序时，可以在**类路径**上列出所有要使用的JAR：

```
java -classpath 路径列表或jar包列表  
-jar 要运行的JAR包名.jar
```

```
java -classpath lib/guava-19.0.jar:\  
lib/hibernate-validator-5.3.1.jar:\  
lib/jboss-logging-3.3.0Final.jar:\  
lib/classmate-1.3.1.jar:\  
lib/validation-api-1.1.0.Final.jar \  
-jar MyApplication.jar
```



Java应用程序必须能找到它所依赖的所有JAR包才能运行。运行时，JVM会到JAR包中去加载要用到的类。

如果在类路径或者JAR包中没有找到所需的类，JVM会抛出一个运行时异常。

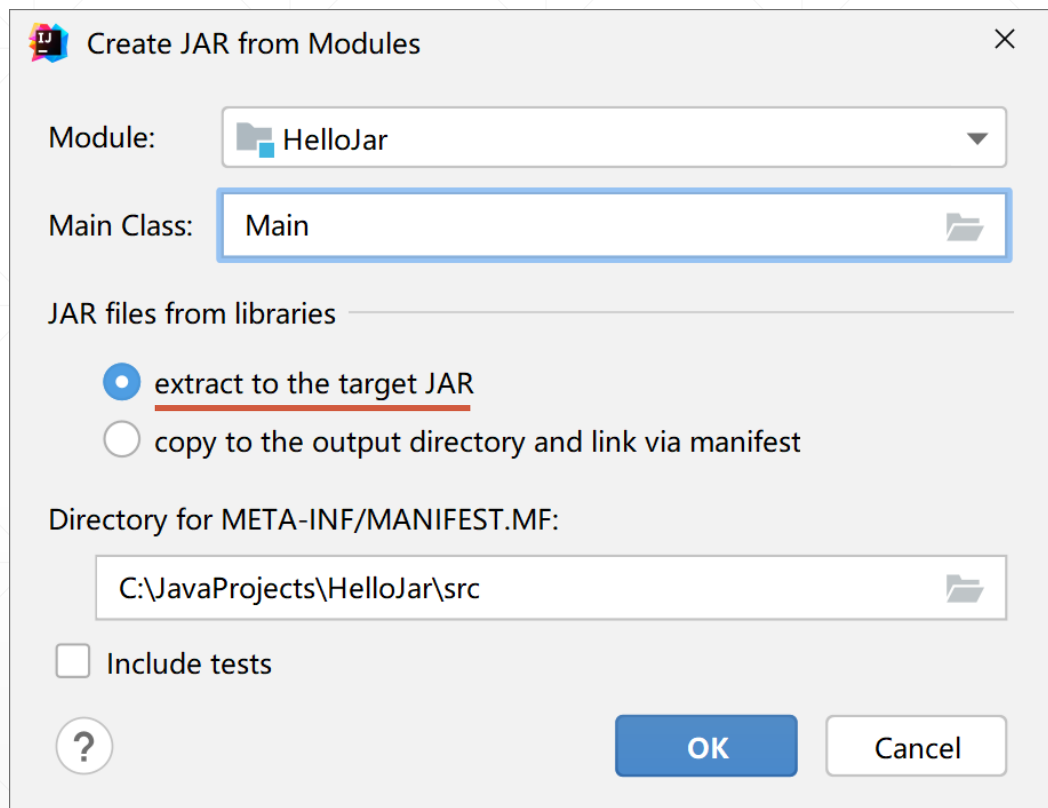


好象我在实际开发中需要运行一个打包好的Java应用程序时，多半就是这样就可以运行它了：

```
java -jar MyJavaApp.jar
```

很少遇到需要指定-classpath参数的场景，为什么会出现这种情况？在什么情形下，运行JAR包必须指定类路径？

原因



不需要指定类路径的JAR包，是一种“fat jar”，它将运行所需的资源全部打包到了单独的一个JAR包中。

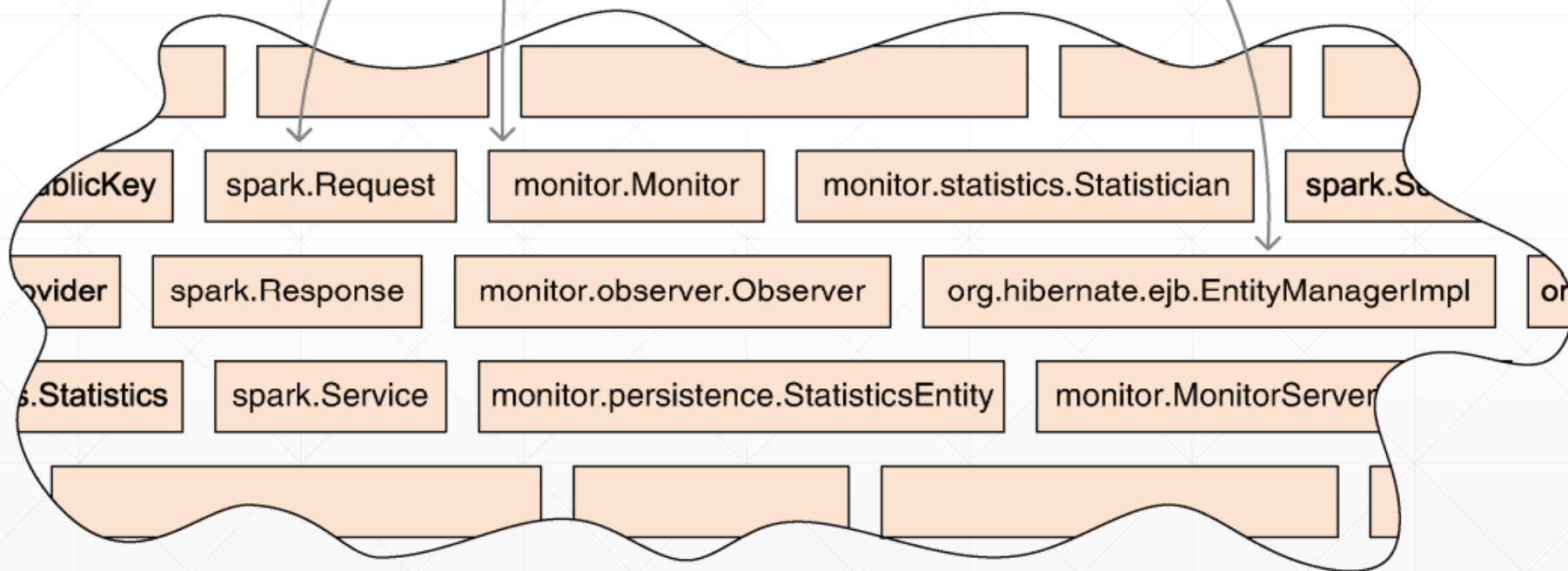
默认情况下，使用IntelliJ IDEA生成JAR包，会自动选中“extract ...”这个选项，将用到的外部JAR包中的.class文件，抽出来打包到自己的JAR包中，这样一来，在运行时，就不再需要它所依赖的外部JAR出场了。



当且仅当要运行的JAR包中没有包容它所依赖的其他JAR包中的类型时，才需要在运行时指定类路径，通知JVM到指定的位置去找特定类型的字节码。

从Jar包中加载类型

从多个JAR包中抽取出来的类，被加载到单个的命名空间中



JVM默认类加载机制引发的“同名类覆盖”现象



在Java程序运行时，JVM会从JAR包中抽取所有类构成一张“大表”，然后在这张大表中按顺序查找特定的类型。



因为一个类会从类路径中第一个包含它的JAR中加载，所以这个类会导致所有其他同名类不可用，这被称为**覆盖**。



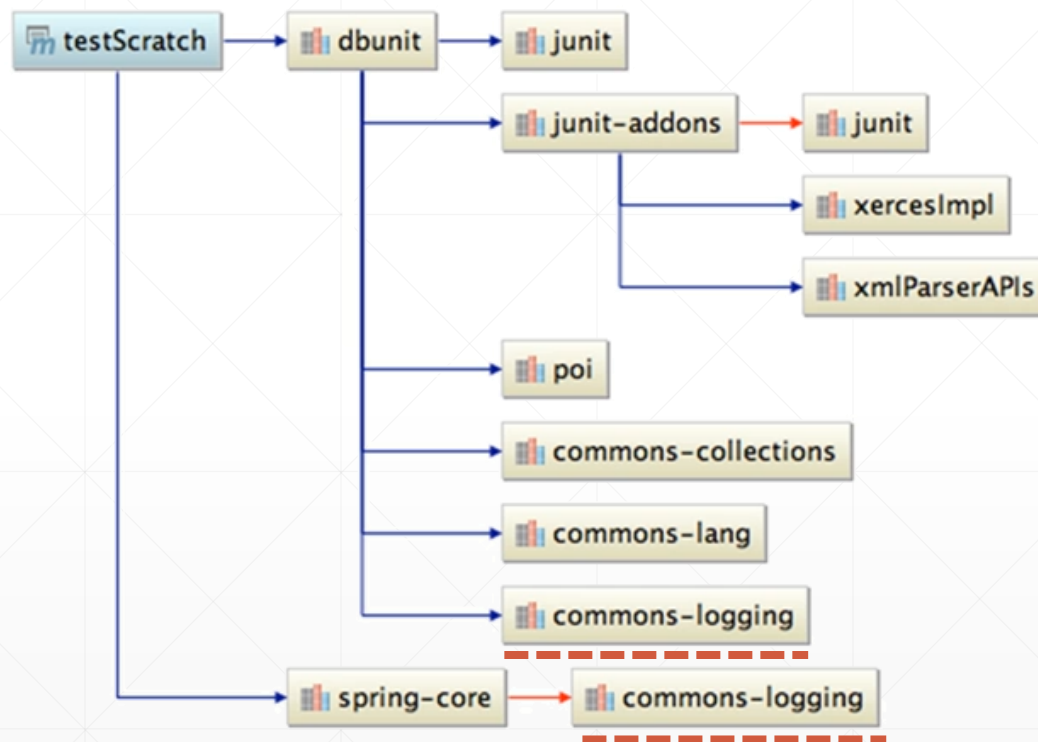
由于在类路径扫描中只有第一个搜索到的实例（它覆盖了所有其他实例）会被加载，所以JAR文件的扫描顺序决定了哪些代码会被执行。

同一项目不同JAR版本间的冲突

”

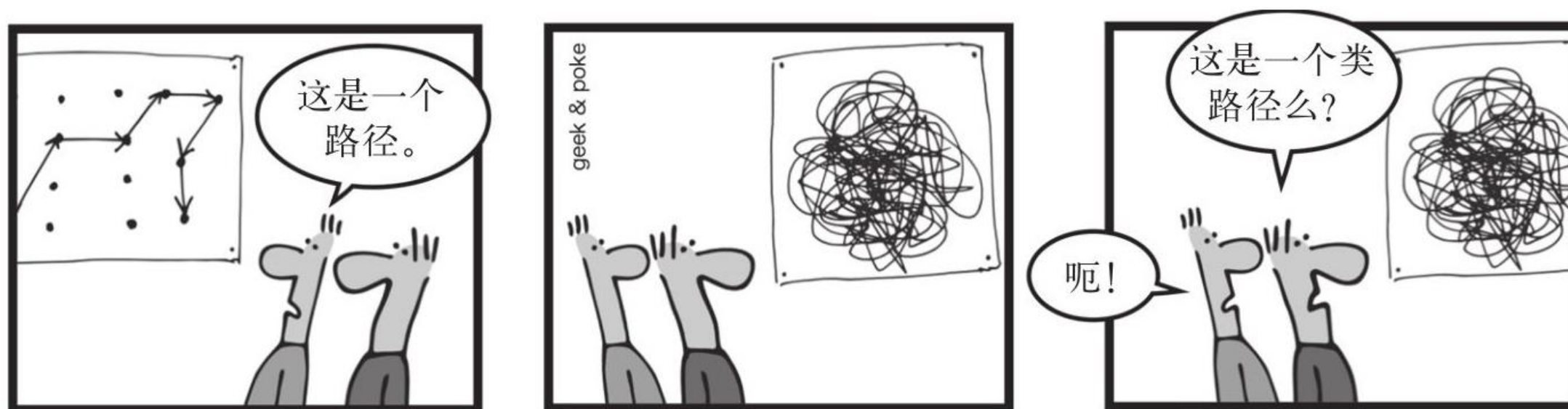
当两个类库分别依赖第三方类库的两个不兼容版本时，就会发生版本冲突。可能会导致加载意料之外的版本，或者是两个版本先后被混合加载。

默认情况下，诸如Maven这样的构建系统，会选择组件树中“最近的(closest)”的版本加载。



类路径地狱 (classpath hell)

当类路径集合包容太多的文件夹，太多的JAR文件，管理和维护它们，是一场“噩梦”



——图摘自《Java 9模块化开发》

JAR地狱 (JAR hell)



某个JAR依赖包缺失



某个JAR依赖包被多次包含，并且很可能以不同版本的形式被多次包含



JAR包依赖的传递性导致出现循环依赖



外部代码使用反射等技术访问JAR包中的非公有类型代码，导致升级和维护困难。

较差的启动性能



如果某程序在运行时依赖于某个类，JVM的“类加载器（Class Loader）”是没办法知道它来自于哪个JAR的，所以它必须对类路径中的所有JAR都执行一次线性扫描，才能找到并加载这个类。



又比如，在程序运行时如果想知道有哪些类添加了特定的“注解（annotation）”，就需要调用反射技术查询类路径中的所有类。

由于JVM采用了“延迟加载类型”的策略，可能需要多次扫描类路径中的JAR包，这些都是耗时操作，所以，许多人抱怨“Java程序”启动慢、占用系统资源多.....

复杂的类加载机制.....



Java允许开发者自由添加额外的类加载器，还可以将类加载由一个加载器委托给另一个.....



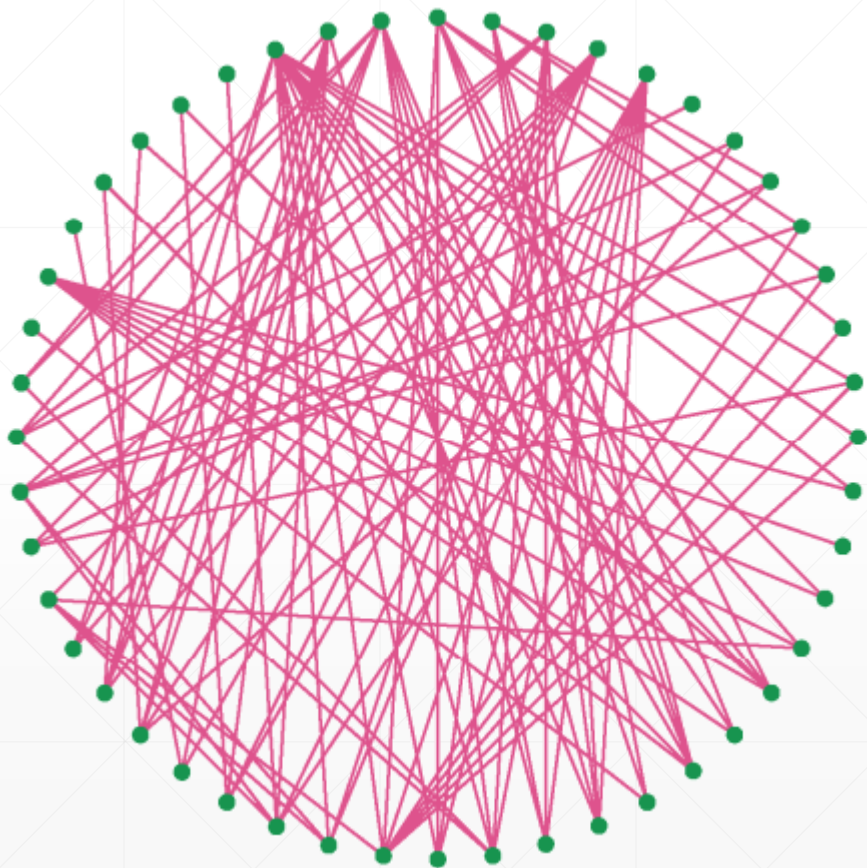
于是，在Java技术演进历史上，就有不少开发者自定义了类加载器（Class Loader），以自己的方式加载类，以便实现某种特定的功能。



许多组件容器（比如Tomcat这个著名的Web Server）定义了自己的Class Loader，这就使得Java应用的类加载方式多种多样，这就给Java技术的进一步演进，带来了沉重的负担，保证兼容性很麻烦，不能轻易改动。



复杂的组件依赖管理



实际开发中，使用的JAR包通常数量很大，这些Jar包之间存在着复杂的依赖关系，维护起来相当困难。

为了解决这个问题，人们先后发明了诸如Maven和Gradle这样的构建管理工具，自动管理一个项目所引用的Jar包之间的依赖关系，但这些工具只能在开发与编译期间起作用，对运行时出现的各种情况，它们无能为力，还是没有从根本上解决问题。

Java平台自己也有问题.....

庞大的运行时环境



随着版本的演化，JRE功能越来越多，相应地，也变得越来越庞大，一个Java应用即使只使用JRE中很少一部分的功能，也不得不配上一个完整的JRE。因为在Java 8之前，人们无法仅安装Java 运行时环境（JRE）的一个子集。因此所有的Java安装往往都有一些不必要的功能组件，比如只跑在服务器上的Java应用，却不得不将Swing组件也打包部署。



给Java应用程序附加不必要的组件，不仅占用更多的存储空间，也拖慢了程序的启动速度和响应性能，因为现在需要在更多的JAR包中去查找与加载特定的类型，影响运行时性能。

Java平台自己也有问题.....

安全性问题

✓ Java在过去经历过相当多的安全漏洞。这些漏洞都有一个共同的特点：不知何故，攻击者可以绕过JVM的安全机制并访问JDK中那些原先不许可外部调用的敏感类型。

✓ 从安全的角度来看，应该要对危险的内部类型进行强封装；同时，减少运行时可用类的数量以尽量降低可能的外部攻击点，从而提升系统的安全性。



<https://www.osgi.org>

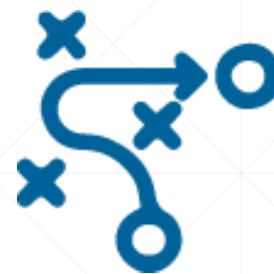
Java模块化的初次尝试

目前最古老且最知名的模块系统是OSGi。它通过在OSGi容器中运行一种名字叫作“bundle”的组件来提供程序运行时的模块化技术特性。

OSGi功能十分强大，但也相对复杂。这种复杂性，导致其在业界的推广受阻，因为它所提供的灵活性，其实并没有那么多的应用场景。



无数的事实说明，过于复杂的技术，其推广往往很成问题，这就为那些能做同样的事情，但比较简单易学的“后来者”，带来了“逆袭”的机遇。



JDK 9中引入了新的模块管理系统，以便解决前面提到的各种问题.....

模块化的目标与好处



JDK 9引入模块化技术之后，可以得到以下的好处：



启动时就能发现模块的缺失问题而立即报错，不需要等到运行后，程序走到特定流程时才触发错误。



能明确地控制代码的可访问性，不希望开放的代码，外部就无法访问。



由于提前知道了许多信息，因此，有可能对其进行进一步的优化。



可定制运行时环境，目标计算机上不再需要安装完整版本的JRE，节约了磁盘空间。

下一讲



从单体应用到模块化应用